

EXPERIMENT 31 — PATTERN MATCHING

Aim

To write and execute a Prolog program to implement **pattern matching using unification**.

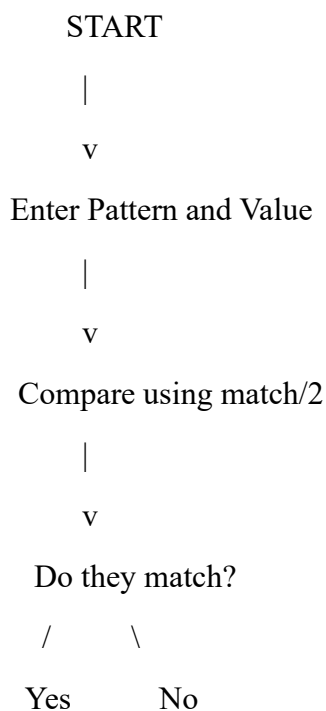
Objective

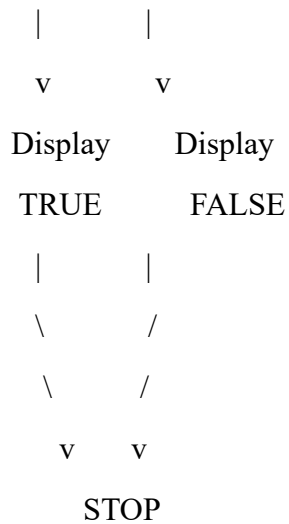
- To understand pattern matching in Prolog.
- To understand the concept of unification.
- To compare two patterns and determine whether they match.
- To retrieve values using variables.

Algorithm

1. Start the program.
2. Define the match/2 predicate.
3. Pass a pattern and a value as arguments.
4. Prolog compares both arguments using unification.
5. If both patterns match, return true.
6. If they do not match, return false.
7. If a variable is present, bind the variable to the matching value.
8. Stop.

Flowchart





Prolog Program

match(Pattern, Pattern).

This single rule performs pattern matching using **Prolog unification**.

Queries

Query 1

?- match(hello, hello).

Output

true.

Query 2

?- match(cat, dog).

Output

false.

Query 3

?- match(apple, X).

Output

X = apple.

Query 4

?- match(X, 100).

Output

X = 100.

Sample Output

```
?- match(hello, hello).
```

```
true.
```

```
?- match(cat, dog).
```

```
false.
```

```
?- match(apple, X).
```

```
X = apple.
```

```
?- match(X, 100).
```

```
X = 100.
```



```
SWI-Prolog console
File Settings Tools Debug Help
Welcome to SWI-Prolog (threaded, 64 bits, version 10.0.2)
SWI-Prolog comes with ABSOLUTELY NO WARRANTY. This is free software.
Please run ?- license. for legal details.

For online help and background, visit https://www.swi-prolog.org
For built-in help, use ?- help(Topic). or ?- apropos(Word).

72 ?- consult(['c:/Users/sadha/OneDrive/Desktop/exp-31.pl']).
true.

73 ?- match(hello, hello).
true .

74 ?- match(cat, cat).
true ▲
```

Result

Thus, the Prolog program for **pattern matching using unification** was successfully implemented and executed.